

README

ota-rollout-engine

Field	Value
Revision	2
Created	2026-06-07
Status	implemented
Part of	Helix OTA
Module path	github.com/HelixDevelopment/ota-rollout-engine
Language	go (1.26)
License	Apache-2.0

Overview

ota-rollout-engine (package rollout) is an OS-agnostic, **I/O-free** staged-rollout engine. It turns an all-at-once delivery into a sequence of percentage-gated phases, each guarded by success/error thresholds and a duration, and decides — from a health verdict supplied by the caller — whether to **HALT, ADVANCE, HOLD, or COMPLETE** a rollout, enforcing the safety invariant that **halt wins over advance** when both could trigger in the same window ([doc.go](#), [decide.go](#)).

It contains **no HTTP, database, disk, network, or OS specifics**. All side effects flow through two ports — `StoragePort` (persist/load state) and `Clock` (current time, so durations are deterministically testable). Telemetry is never ingested here: the caller computes a `HealthVerdict` from its own pipeline (e.g. [ota-telemetry-schema](#)) and hands it to `Engine.Evaluate`. It depends on the `stdlib` plus [ota-protocol](#) for the shared `DeviceDeploymentStatus` enum only.

Cohort selection is deterministic and stable: a device is in a phase's cohort when $\text{FNV-1a}(\text{deviceId} + 0x00 + \text{deploymentID}) \bmod 100 < \text{cumulativePercentage}(\text{cohort.go})$. Because the hash is fixed and percentages strictly increase across ordered phases, cohort membership only grows as the rollout widens (monotonic growth).

Public API

Engine (engine.go)

- `New(store StoragePort, clock Clock) (*Engine, error)` — both ports must be non-nil (`ErrNilPort`).
- `(*Engine).Create(ctx, deploymentID string, phases []Phase) (State, error)` — validate the plan, persist initial `StatusPending`.
- `(*Engine).Start(ctx, deploymentID string) (State, error)` — activate the first phase, stamping its start time; idempotent on an already-active rollout.
- `(*Engine).Evaluate(ctx, deploymentID string, v HealthVerdict) (Decision, error)` — apply one verdict to the current phase, persist any transition, return the `Decision`. Idempotent at terminal status.

Plan & state (types.go)

- `Phase` — `Percentage` (cumulative, $[0, 100]$), `SuccessThreshold`, `ErrorThreshold` (fractions $[0, 1]$), `Duration`, `AutoProgress`.
- `Status` enum: `StatusPending`, `StatusActive`, `StatusHalted`, `StatusHeld`, `StatusCompleted` (with `Valid()`).
- `State` — persisted plan + cursor (`DeploymentID`, `Phases`, `CurrentPhase`, `Status`, `PhaseStartedAt`, `UpdatedAt`); `Clone()`, `Phase()` (`Phase`, `bool`).
- Plan-validation sentinels: `ErrNoPhases`, `ErrEmptyDeploymentID`, `ErrPercentageRange`, `ErrPercentageNotMonotonic`, `ErrThresholdRange`, `ErrDurationNegative`, `ErrFinalPercentageNot100`.

Verdict & decision (verdict.go)

- `HealthVerdict` — caller-supplied current-phase summary: `SuccessRate`, `ErrorRate` (fractions $[0, 1]$), `PostBootHealthFailed`. Range sentinel `ErrVerdictRange`.
- `Action` enum: `ActionHalt`, `ActionAdvance`, `ActionHold`, `ActionComplete`.
- `Reason` enum: `ReasonErrorThreshold`, `ReasonPostBootFailed`, `ReasonSuccessThreshold`, `ReasonWindowOpen`, `ReasonWindowExpired`, `ReasonAutoProgressOff`, `ReasonNoActivePhase`.
- `Decision` — `Action`, `Reason`, `Status`, and `DeviceStatus` `otaprotocol.DeviceDeploymentStatus`.

Ports & cohort (ports.go, cohort.go)

- `StoragePort` interface (`Load`, `Save`) with `ErrNotFound` sentinel; `Clock` interface (`Now`); `NewSystemClock()` `Clock`.
- `InCohort(deviceID, deploymentID string, cumulativePercentage int) bool` — deterministic, monotonic cohort membership.

Usage

```
package main

import (
    "context"
    "fmt"
    "time"

    rollout "github.com/HelixDevelopment/ota-rollout-engine"
)

func main() {
    // store is your StoragePort implementation (in-memory map,
    // SQL, KV...).
    var store rollout.StoragePort = newMyStore()

    eng, err := rollout.New(store, rollout.NewSystemClock())
    if err != nil {
        panic(err)
    }

    ctx := context.Background()
    phases := []rollout.Phase{
        {Percentage: 10, SuccessThreshold: 0.95, ErrorThreshold:
        0.05, Duration: time.Hour, AutoProgress: true},
        {Percentage: 100, SuccessThreshold: 0.98, ErrorThreshold:
        0.02, AutoProgress: false},
    }
    if _, err := eng.Create(ctx, "dep-1", phases); err != nil {
        panic(err)
    }
    if _, err := eng.Start(ctx, "dep-1"); err != nil {
        panic(err)
    }

    dec, _ := eng.Evaluate(ctx, "dep-1",
        rollout.HealthVerdict{SuccessRate: 0.99, ErrorRate: 0.0})
    fmt.Println(dec.Action, dec.Reason) // advance
        success_threshold_met

    // Deterministic cohort membership:
    fmt.Println(rollout.InCohort("device-42", "dep-1", 10))
}
```

Testing

```
cd submodules/ota-rollout-engine
go vet ./...
go test ./...
```

The suite uses in-memory port fakes (`fakes_test.go`) and covers: cohort determinism, boundaries, monotonic growth, percentage approximation, and per-deployment isolation (`TestInCohortDeterminism`, `TestInCohortBoundaries`, `TestInCohortMonotonicGrowth`, `TestInCohortApproximatesPercentage`, `TestInCohortDeploymentIsolation`); the exhaustive pure decision table (`TestDecide`); plan validation and persistence (`TestCreateValidation`, `TestCreateValidPersists`); start idempotency/terminal guards (`TestStartIdempotent`, `TestStartFromTerminalErrors`); the full phase progression with a fake clock, advance-resets-phase-clock, window hold/expiry, post-boot abort, and the **halt-wins** invariant end-to-end (`TestEvaluateFullProgression`, `TestEvaluateAdvanceResetsPhaseClock`, `TestEvaluateWindowHold`, `TestEvaluateWindowExpiredHolds`, `TestEvaluatePostBootAbort`, `TestEvaluateHaltWinsViaEngine`); terminal idempotency (`TestEvaluateHaltIsIdempotent`, `TestEvaluateCompleteIsIdempotent`); and port/state error paths (`TestSaveErrorPropagates`, `TestLoadInvalidStatus`, `TestStateClone`).

Reusable building brick

This is a **reusable, independently versioned** Helix OTA building brick (Helix Constitution §11.4.28 — submodules-as-equal-codebase). Consume it via its module path `github.com/HelixDevelopment/ota-rollout-engine`. It is a pure engine over a storage port + a clock port, reusable for any fleet rollout. Universal constitution rules are inherited via this repo's `CLAUDE.md` / `AGENTS.md` (`## INHERITED FROM Helix Constitution`).

Mirrors

- GitHub: <https://github.com/HelixDevelopment/ota-rollout-engine>
- GitLab: <https://gitlab.com/helixdevelopment1/ota-rollout-engine>